

DNS Security 2

Cloud Security

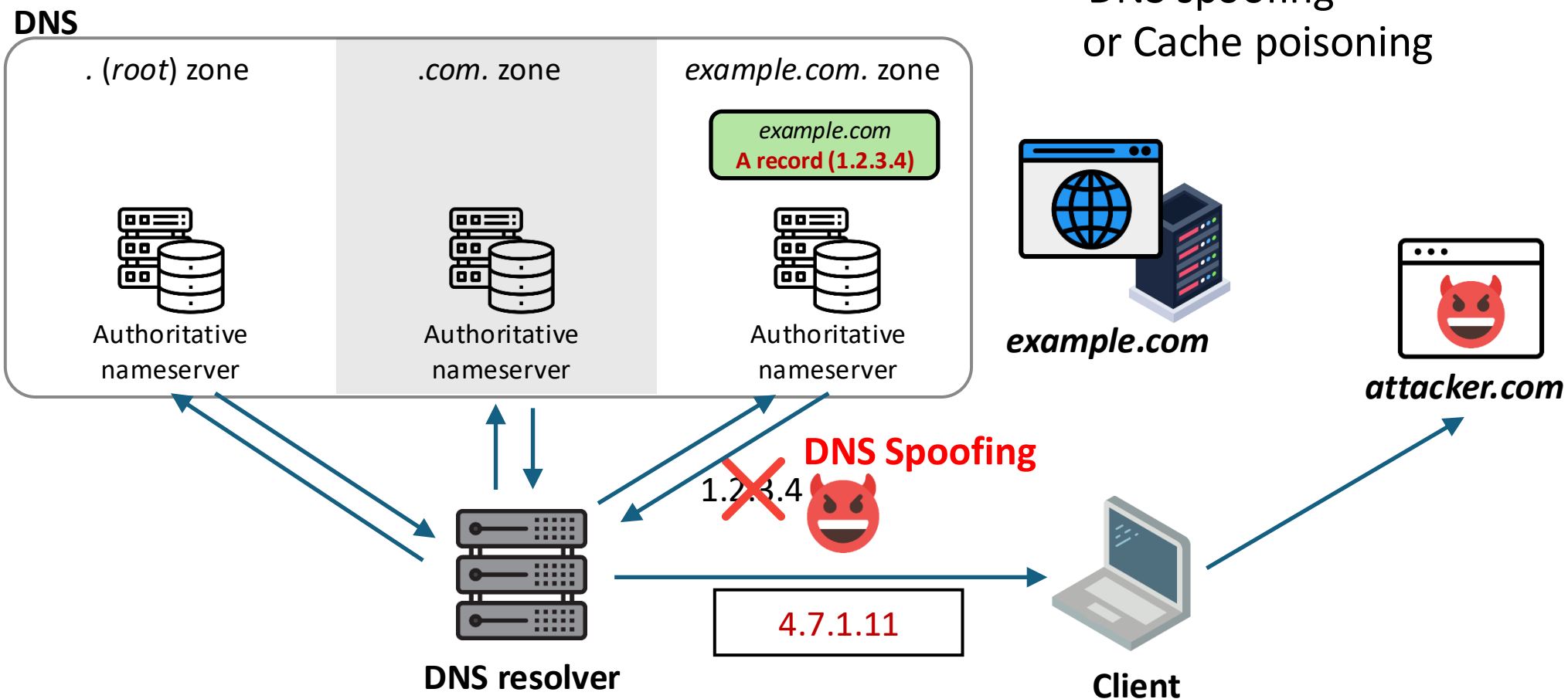
CSE4059

DNS Security

- DNS **does not consider security** in its initial design
 - No integrity check, no authentication

Vulnerable to attacks like

- DNS spoofing
- or Cache poisoning



DNS Security Extensions (DNSSEC)

- DNS protocol extensions for authenticating DNS data using digital signatures
- A DNSSEC-signed domain uses public/private key pairs
 - Private key: used to sign DNS RRsets
 - Public key: published in DNS
- A validator verifies the signatures
 - Checks data **origin authentication** and **integrity**
 - A validator is commonly a recursive resolver, but validation can also be performed by a client
- DNSSEC does not provide confidentiality
 - DNS messages are not encrypted

DNS Security Extensions (DNSSEC)

- New DNS record types were added for DNSSEC
- Three key DNSSEC record types

DNS Record	Description
DNSKEY	- Contains the public keys used by a zone - The corresponding private keys are used to sign DNS records (RRsets)
RRSIG	- Contains digital signatures of DNS records, generated using the corresponding private key
DS	- Contains a digest (hash) of a DNSKEY - Published in the parent zone to establish a chain of trust from the root to the child zone

Resource Record Set (RRset)

```
└─ dig @1.1.1.1 google.com A
; <<>> DiG 9.10.6 <<>> @1.1.1.1 google.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR,
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHOR
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 1232
;; QUESTION SECTION:
;google.com.                IN      A
;; ANSWER SECTION:
google.com.      127      IN      A      172.217.211.139
google.com.      127      IN      A      172.217.211.101
google.com.      127      IN      A      172.217.211.113
google.com.      127      IN      A      172.217.211.100
google.com.      127      IN      A      172.217.211.138
google.com.      127      IN      A      172.217.211.102
```

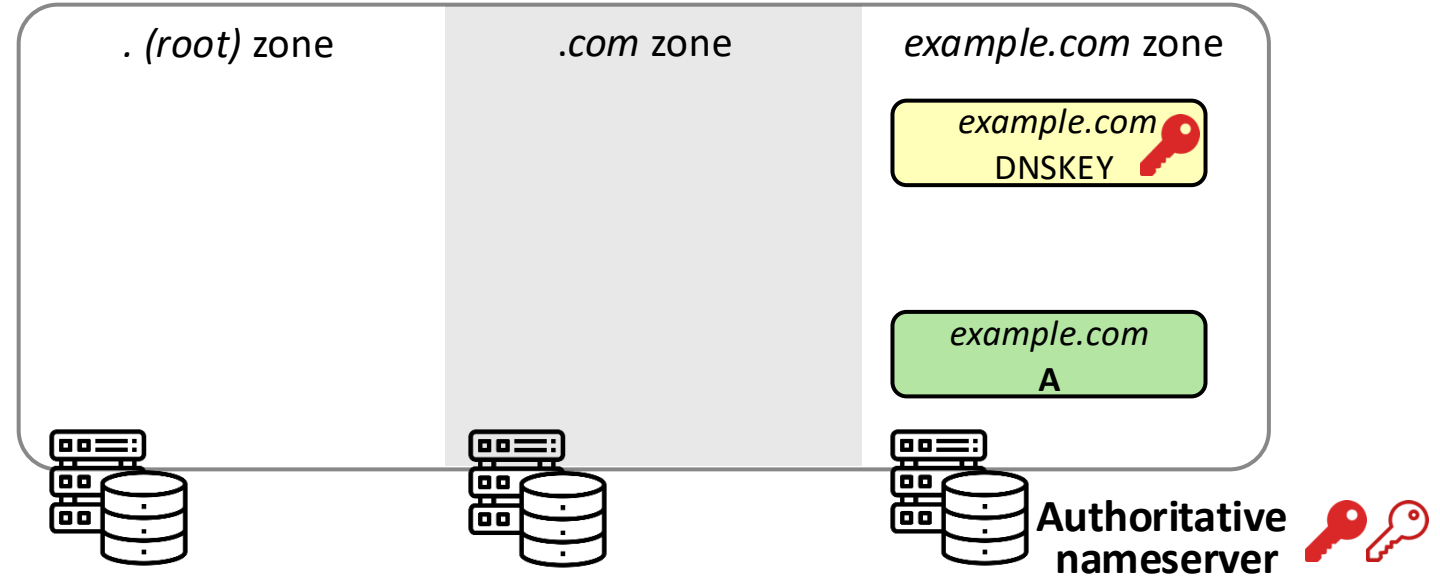
```
└─ dig @1.1.1.1 facebook.com A
; <<>> DiG 9.10.6 <<>> @1.1.1.1 facebook.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 47646
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 1232
;; QUESTION SECTION:
;facebook.com.              IN      A
;; ANSWER SECTION:
facebook.com.      1      IN      A      57.144.44.1
```

Domain-side DNSSEC Deployment

1. Publish public keys as **DNSKEY** records

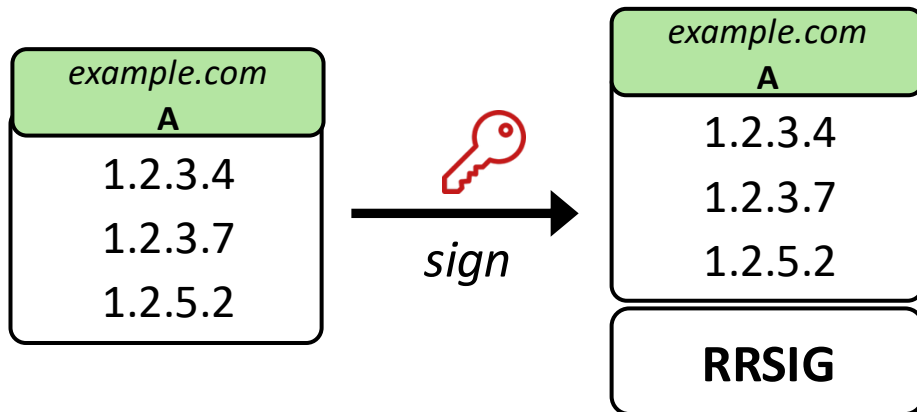
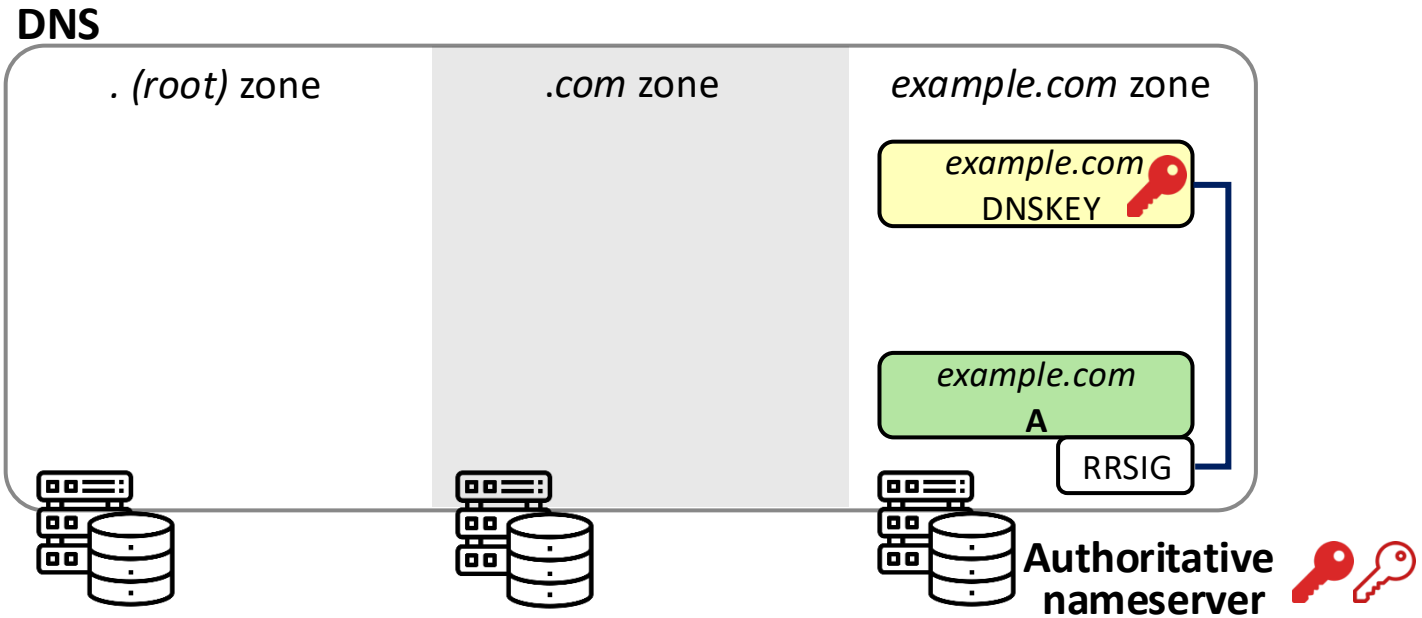


DNS



Domain-side DNSSEC Deployment

1. Publish public keys as **DNSKEY** records 
2. Generate **RRSIG** records for DNS RRsets
 - using the private key 
 - the private key is not published in DNS

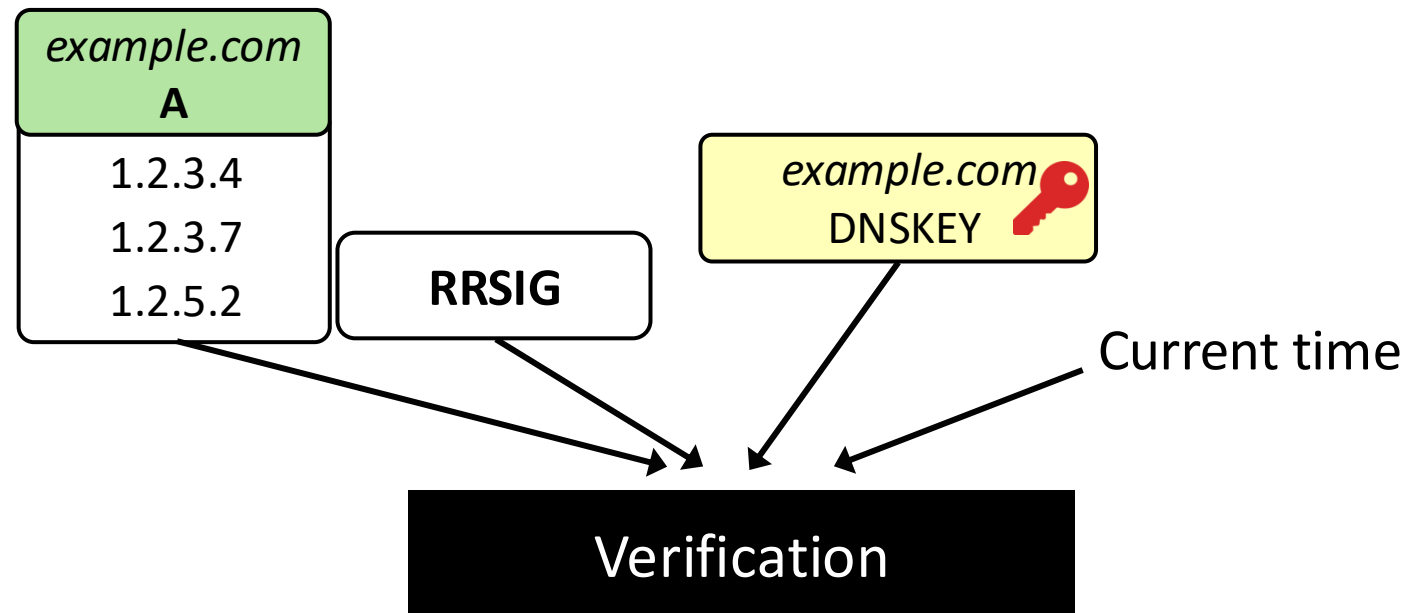


RRSIG

- Carries a signature over an RRset and metadata needed to verify it
 - Ex) Signature Inception and Expiration fields define the signature's validity window

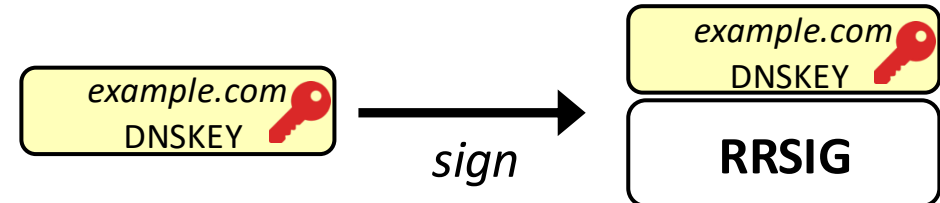
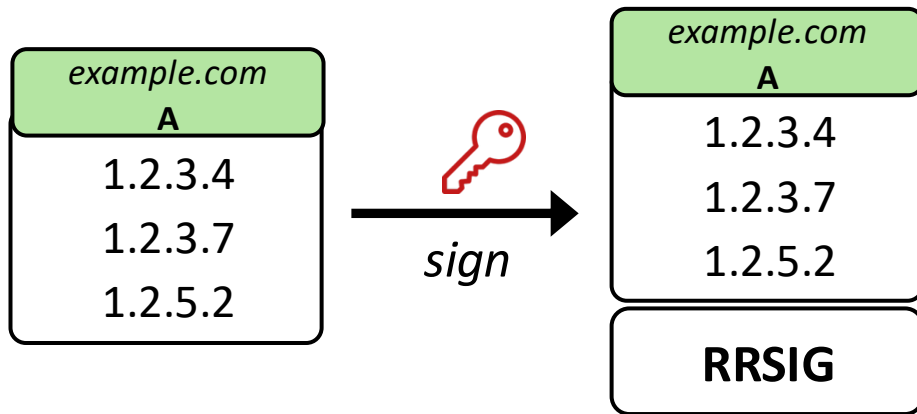
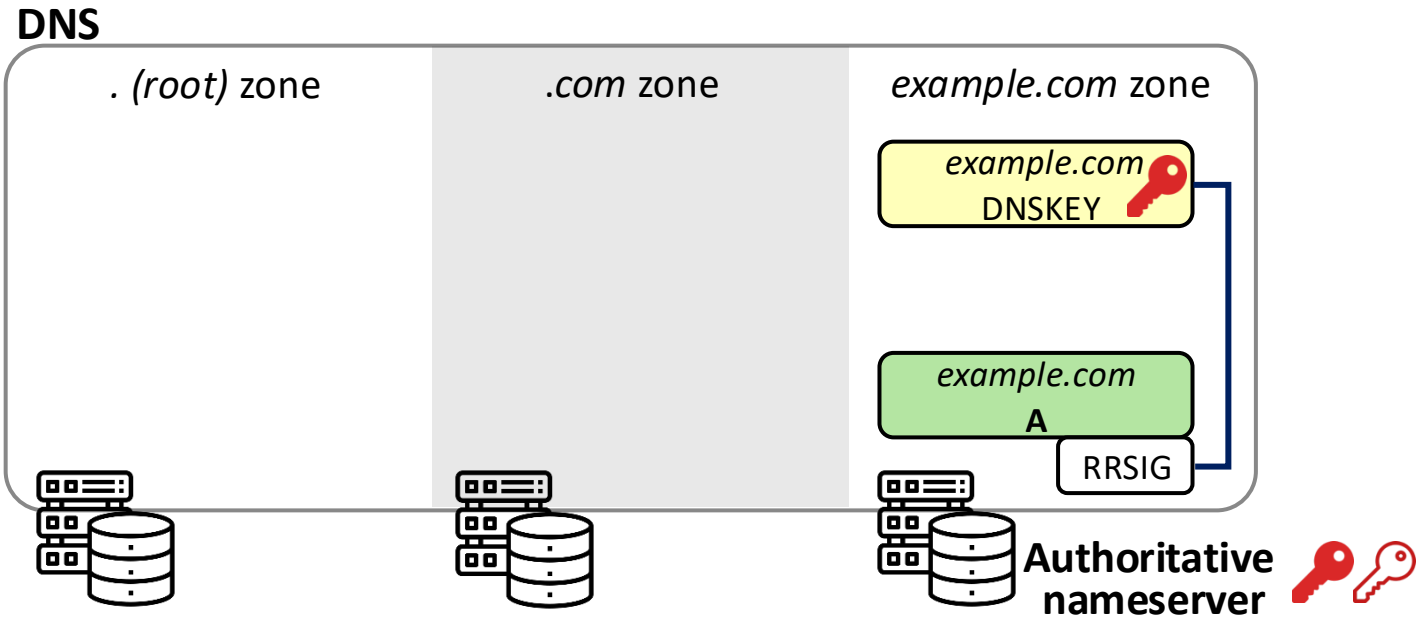
RRset Verification

- Verify the signature using the public key and the received RRset
- Inputs
 - Received RRset, its RRSIG, DNSKEY (public key), and current time
- Check that the RRSIG is valid at the current time



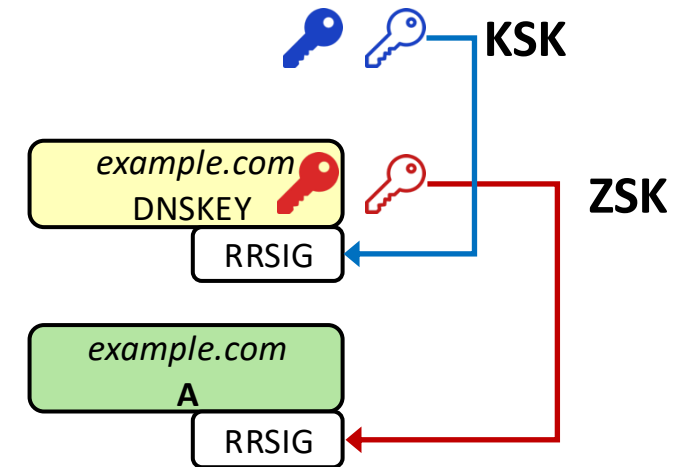
Domain-side DNSSEC Deployment

1. Publish public keys as **DNSKEY** records 
2. Generate **RRSIG** records for DNS RRSets
 - using the private key 



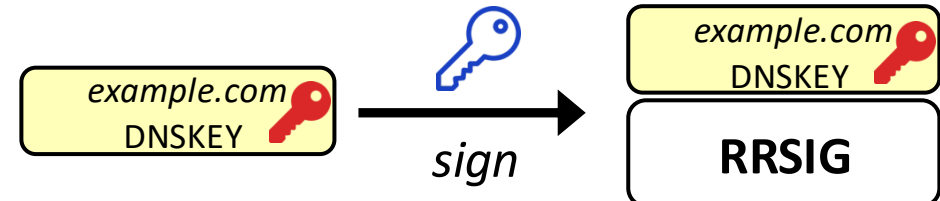
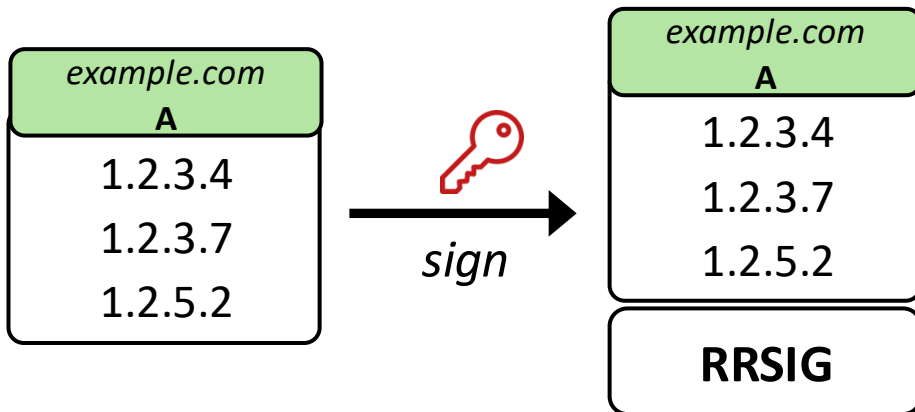
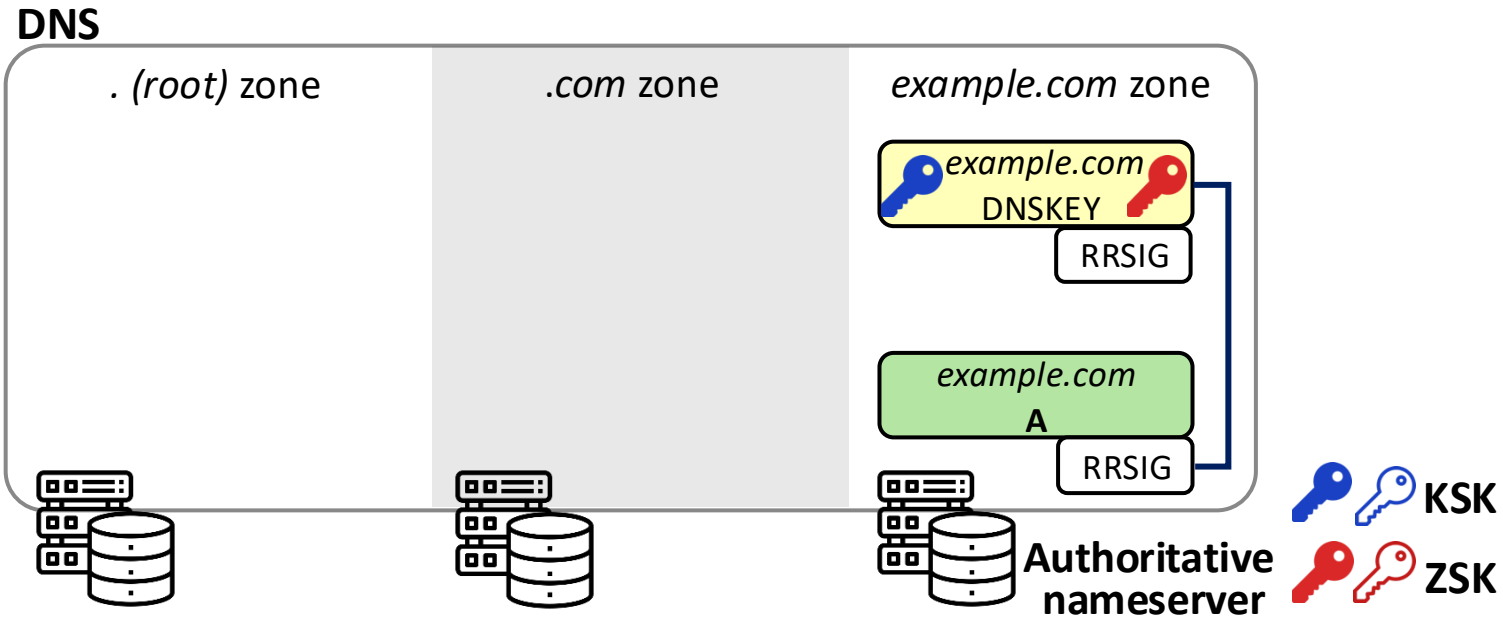
Zone and Key Signing Keys

- DNSSEC utilizes a two-key design
 - Two key pairs
- Zone Signing Key (ZSK)
 - ZSK private key signs all DNS record RRset except DNSKEY
- Key Signing Key (KSK)
 - KSK private key signs the DNSKEY RRset
- Both ZSK and KSK are published as DNSKEY records
- Why?
 - By dividing the keys, a domain can change a ZSK frequently for enhanced security
 - ZSK is more frequently used for generating RRSIGs for all DNS records except DNSKEY



Domain-side DNSSEC Deployment

1. Publish public keys as **DNSKEY** records 
2. Generate **RRSIG** records for DNS RRSets
 - using the private keys  



How Can We Trust Keys

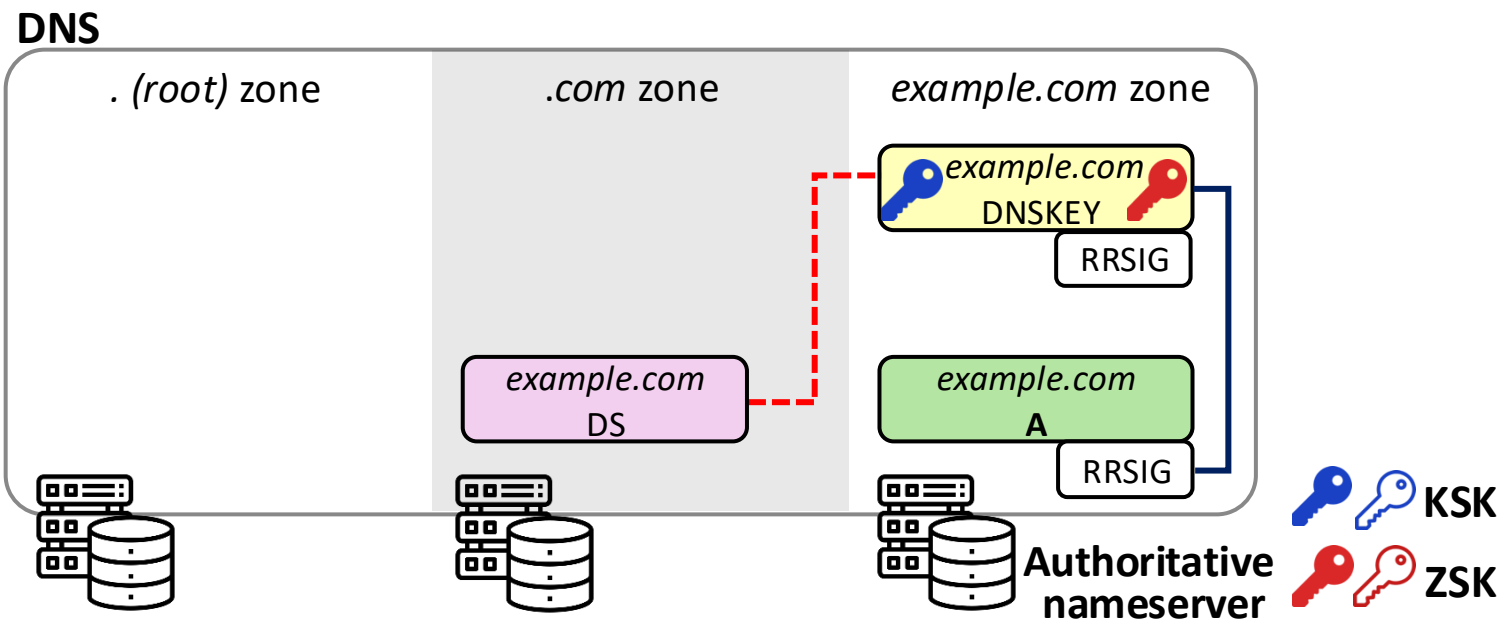
- Let's assume
 - An attacker supplies a false A record, an attacker-controlled DNSKEY, and a matching RRSIG
 - The signature can still verify using the attacker-supplied key
 - But the resolver has no authenticated evidence that this DNSKEY belongs to the expected zone
- Key authentication is required in addition to signature verification

Delegation Signer (DS) Record

- A DS record links a parent zone to a child zone's DNSKEY
- In a typical KSK/ZSK design, the DS record refers to the child zone's KSK DNSKEY
 - Contains a digest derived from the KSK
 - A digest is a fixed-size hash value used to verify the expected key
- The parent zone publishes the DS record for the child
 - Ex) the *.com* zone publishes the DS record for *example.com*
- This link extends the DNSSEC chain of trust from the parent to the child

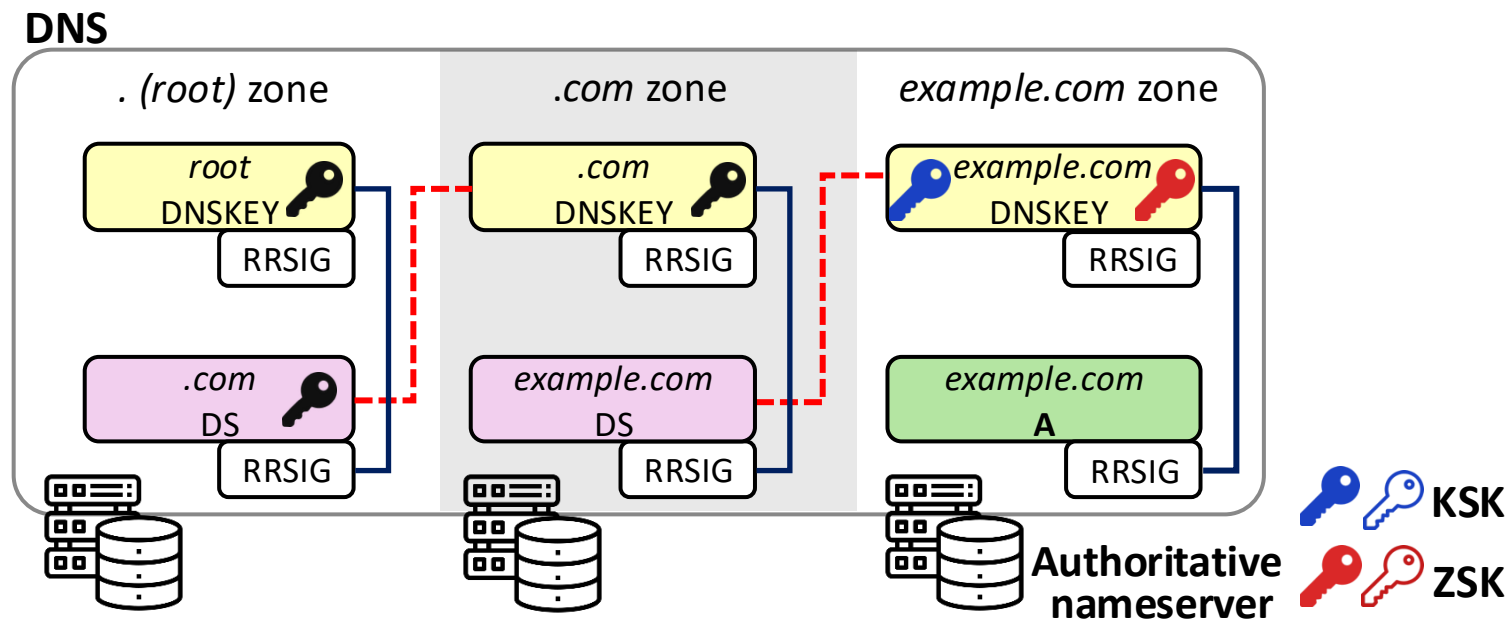
Domain-side DNSSEC Deployment

1. Publish public keys as **DNSKEY** records 
2. Generate **RRSIG** records for DNS RRSets
 - using the private keys  
3. Upload a **DS** record in the parent zone
 - i.e., a digest of KSK 



Domain-side DNSSEC Deployment

1. Publish public keys as **DNSKEY** records 
2. Generate **RRSIG** records for DNS RRSets
 - using the private keys  
3. Upload a **DS** record in the parent zone
 - i.e., a digest of KSK 

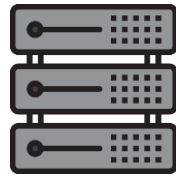
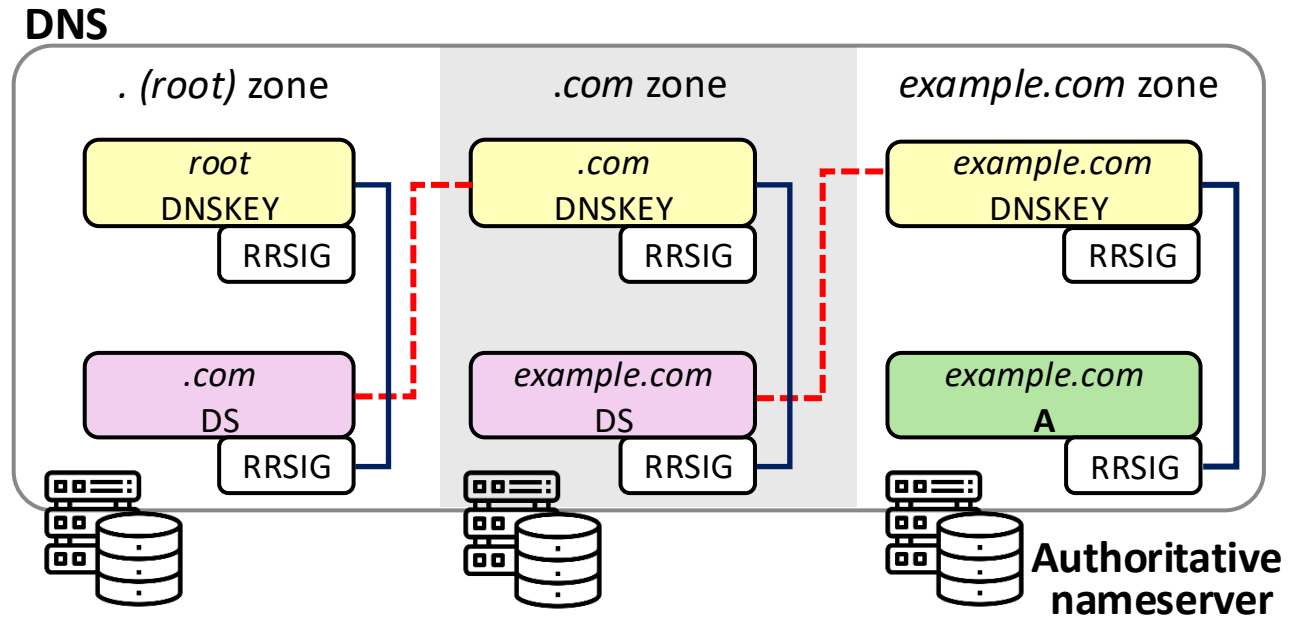


Each level also needs to support DNSSEC
- The same process is repeated

→ Chain of Trust (DNSSEC chain)

DNSSEC validation

- DNSSEC validation is usually performed by resolvers
 - Fetches DNS records together with the required DNSKEY, DS, and RRSIG
 - Validates the chain of trust from the *root* to the target zone
- A client can also perform DNSSEC validation itself

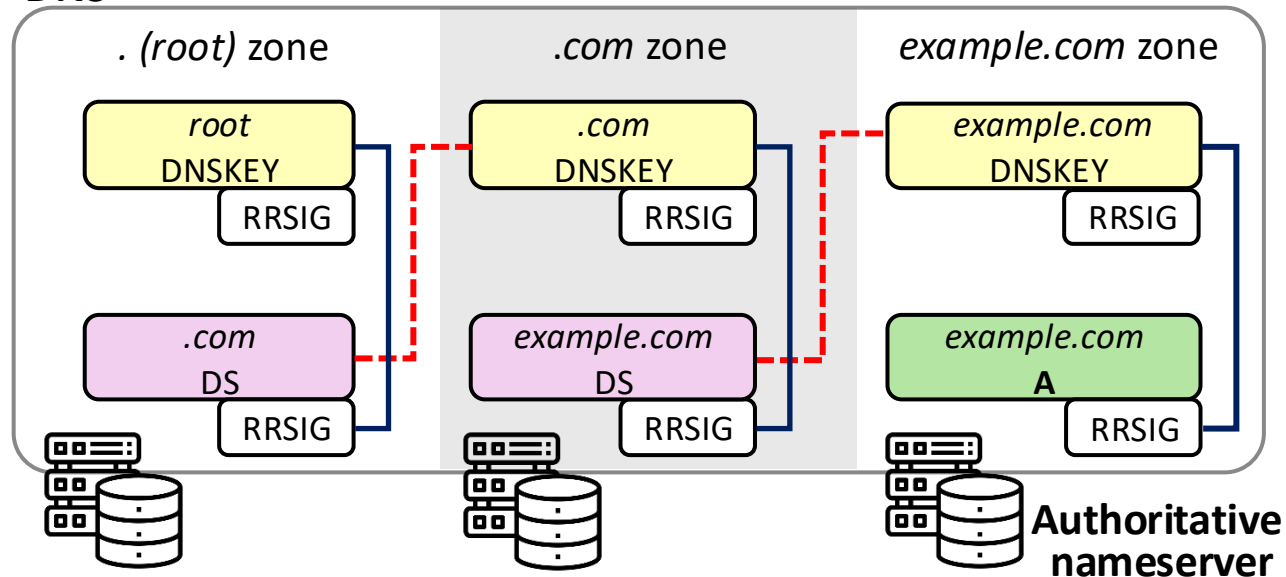


DNS resolver

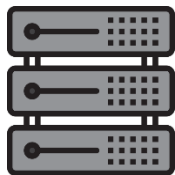
DNSSEC validation

- Root DNSKEY?
 - Validation starts from a configured root trust anchor
 - The *root* KSK is trusted independently by the validator (e.g., resolvers)
 - The trusted *root* KSK is used to validate the root DNSKEY RRset

DNS



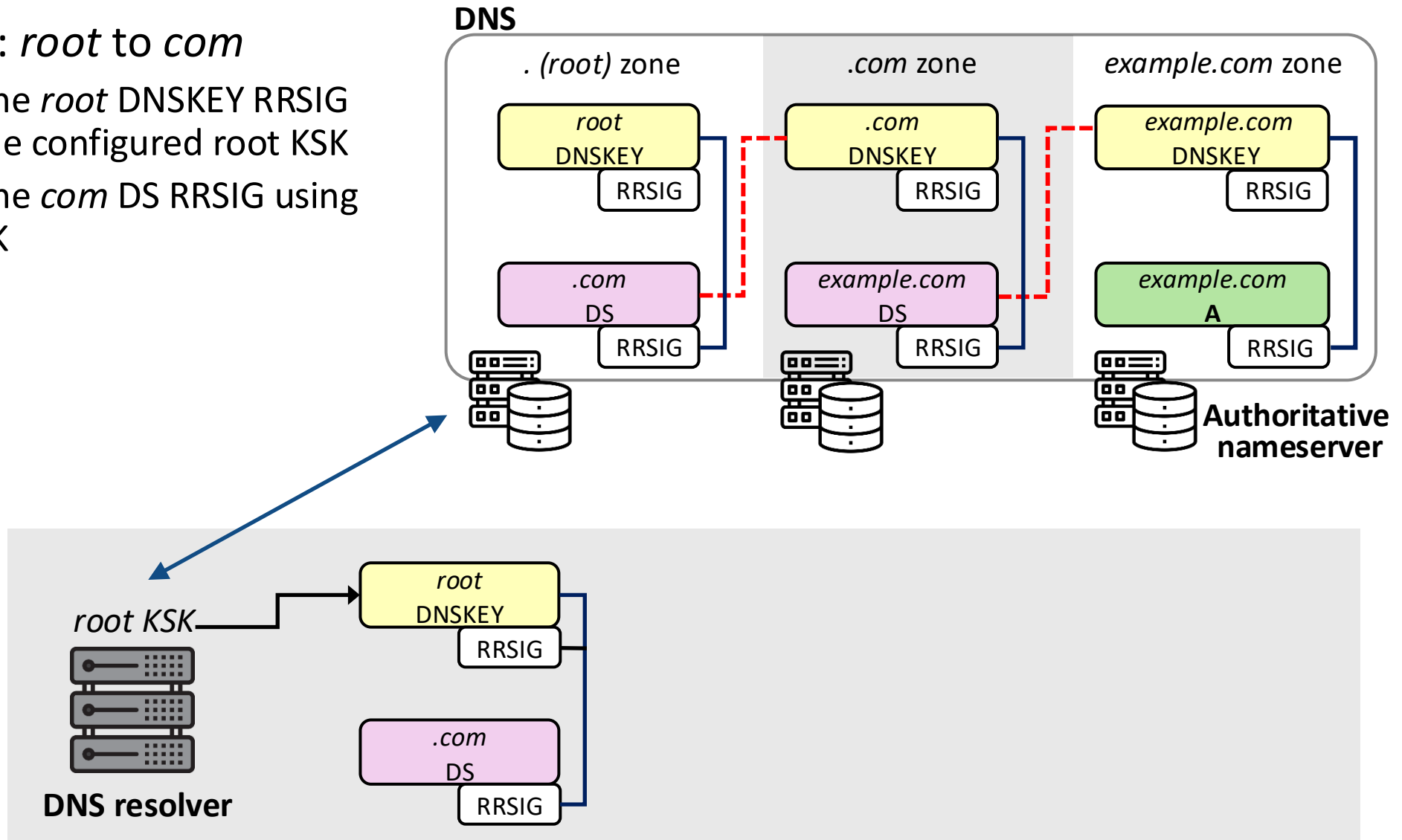
root KSK



DNS resolver

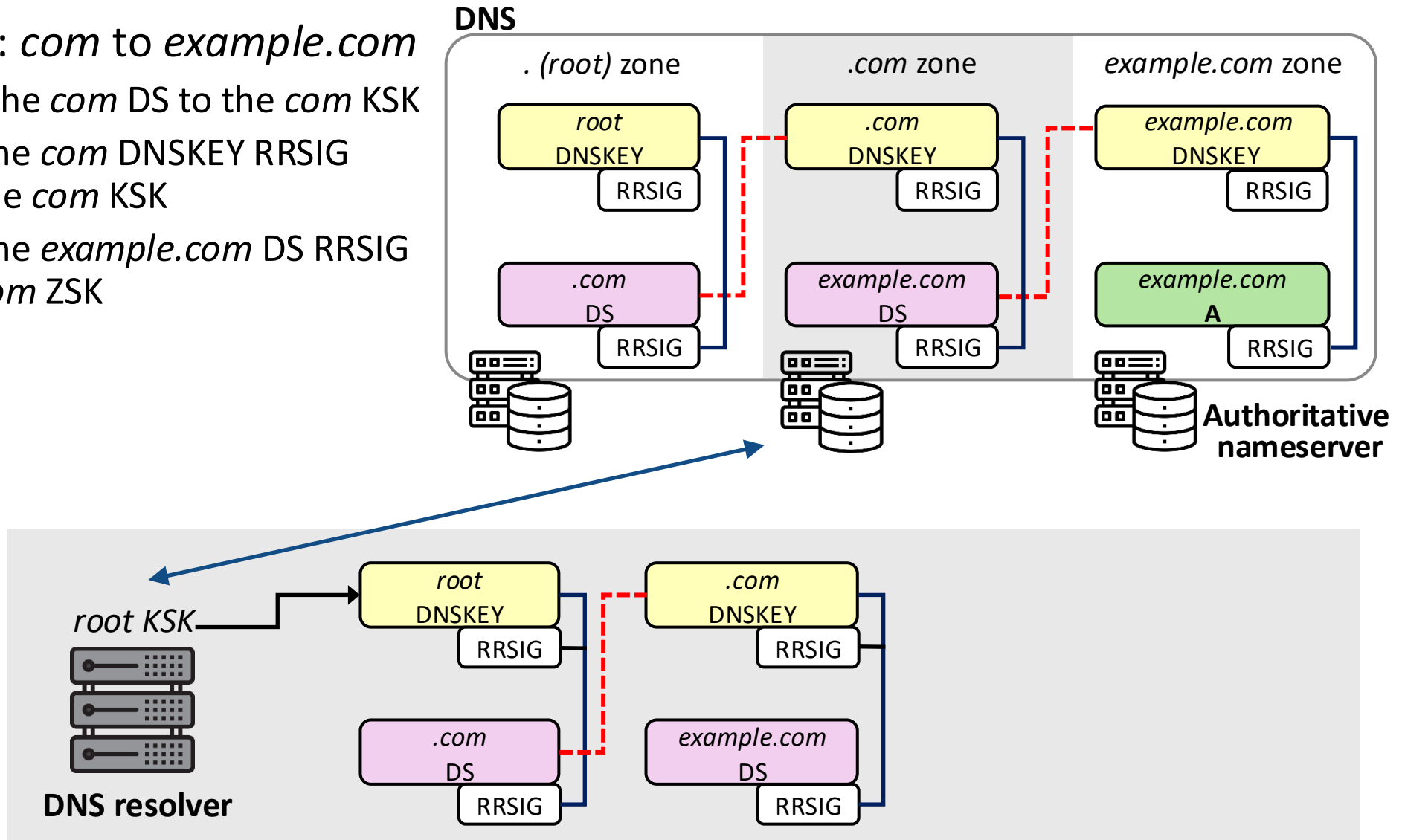
DNSSEC validation

- Validation: *root* to *com*
 - Verify the *root* DNSKEY RRSIG using the configured root KSK
 - Verify the *com* DS RRSIG using *root* ZSK



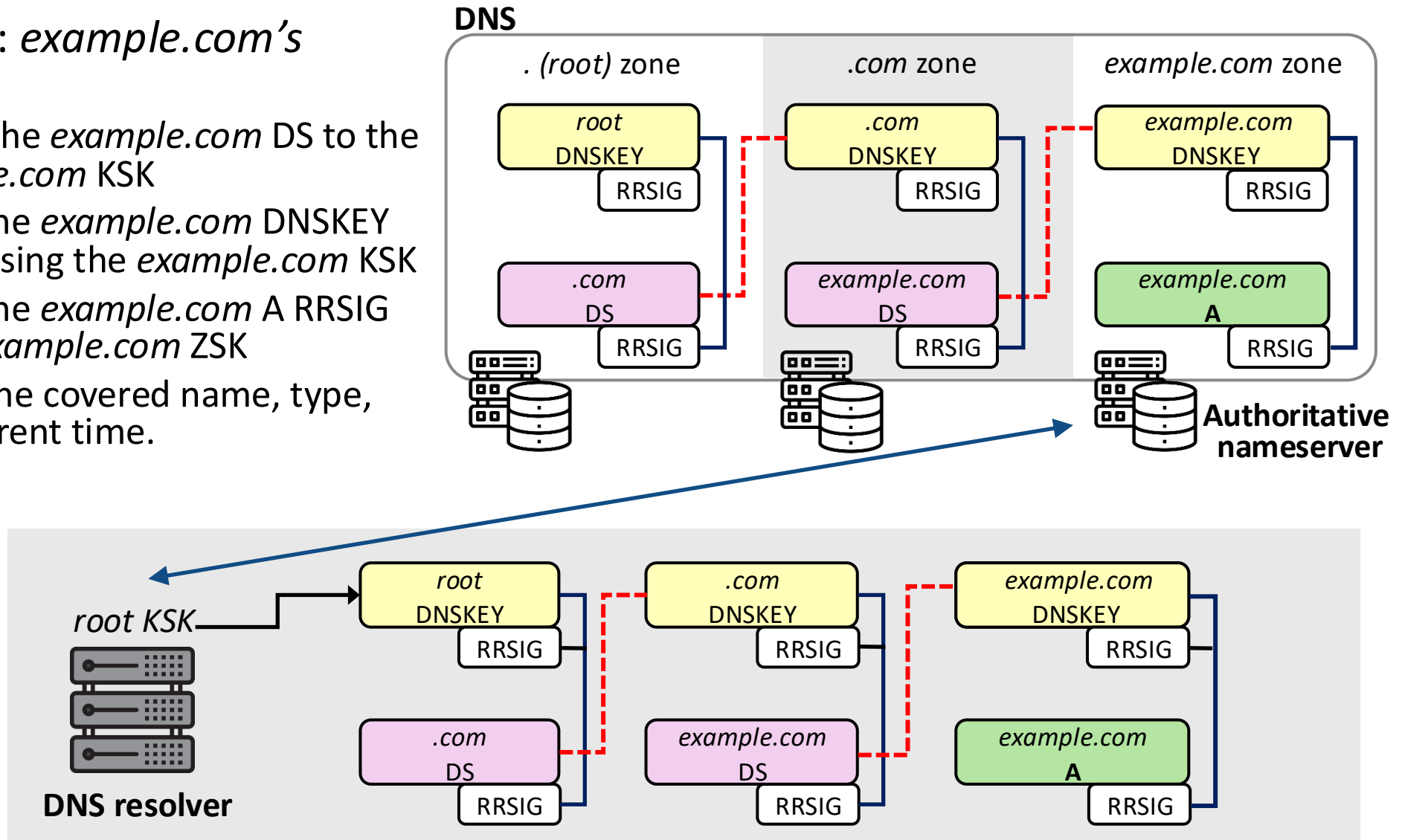
DNSSEC validation

- Validation: *com* to *example.com*
 - Match the *com* DS to the *com* KSK
 - Verify the *com* DNSKEY RRSIG using the *com* KSK
 - Verify the *example.com* DS RRSIG using *com* ZSK



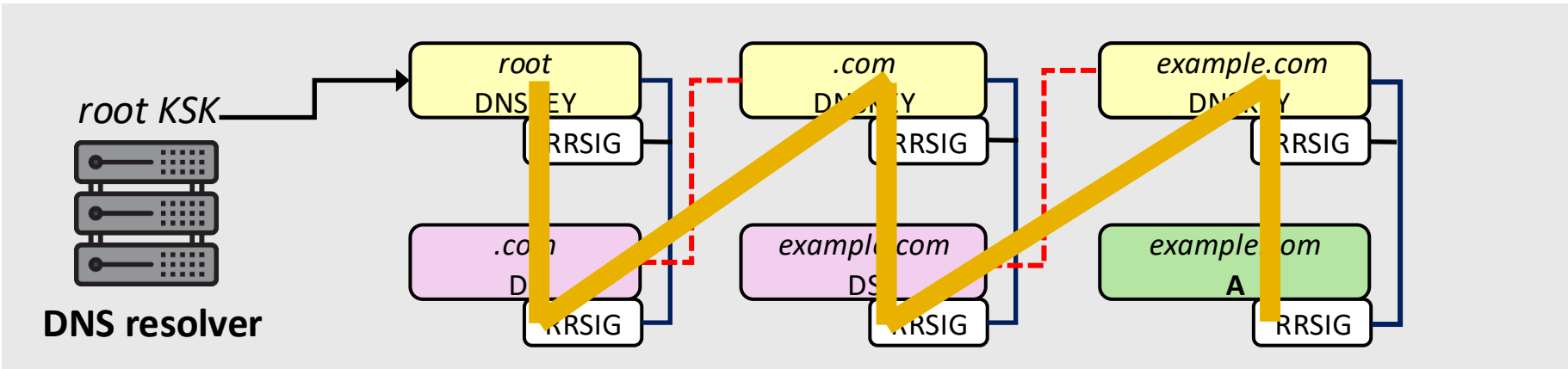
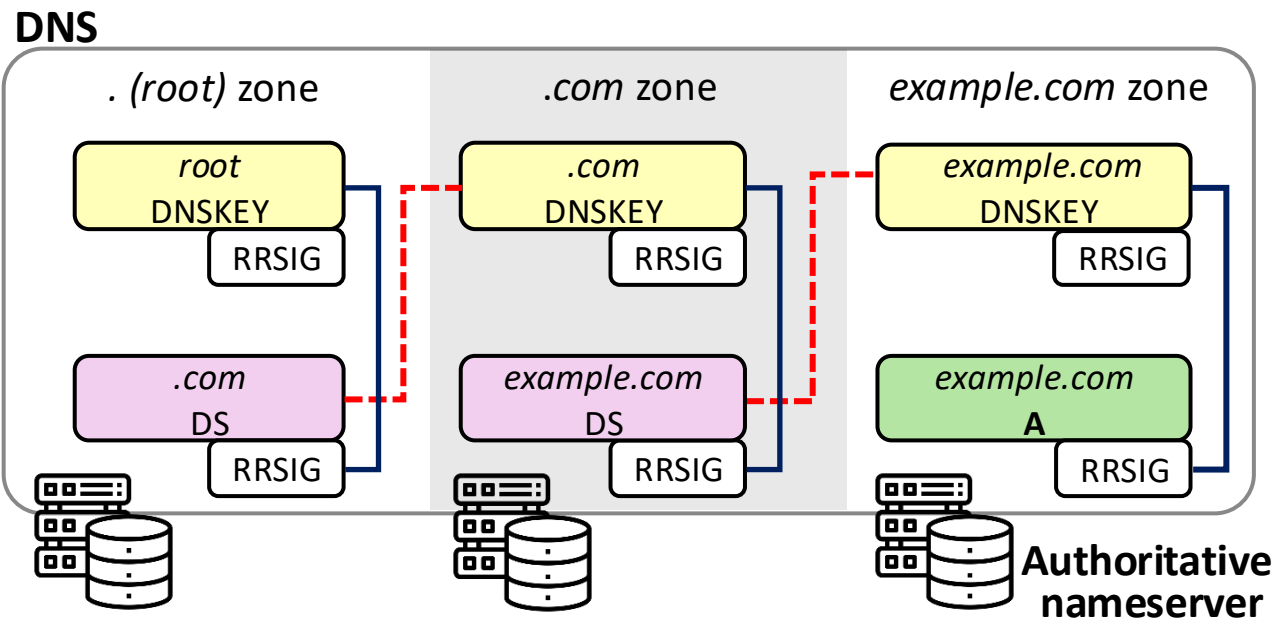
DNSSEC validation

- Validation: *example.com*'s answer
 - Match the *example.com* DS to the *example.com* KSK
 - Verify the *example.com* DNSKEY RRSIG using the *example.com* KSK
 - Verify the *example.com* A RRSIG using *example.com* ZSK
 - Check the covered name, type, and current time.



DNSSEC validation

Chain of Trust (DNSSEC chain)



Validation Case Study: Modified DNS Answer

Input	Original	Modified
A record (IP address)	192.0.2.10	192.0.2.66
RRSIG	Original RRSIG	Original RRSIG
Zone key	Authenticated key	Authenticated key
Time	Within window	Within window
Result	Verifies	Fails

- Assumption
 - Authenticated zone key
 - Valid signature time window
- Result
 - A resolver uses the authenticated key to check the received DNS answer and original signature
 - Signature verification fails
 - DNSSEC validation fails

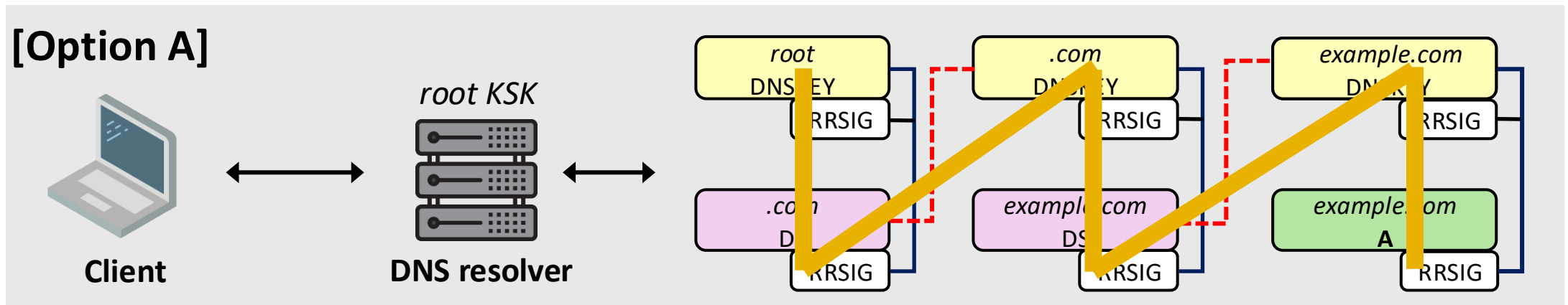
PKI vs. DNSSEC

	PKI	DNSSEC
Starting trust	Trust anchor	Trust anchor
Signed object	Certificate	DNS RRset
Cross-level link	Issuer signature	Signed DS + key match
Final claim	Certificate chain	DNS data

- Both start from independently accepted trust information and authenticate keys before relying on signatures
- Public Key Infrastructure (PKI)
 - Uses CA-signed certificates to bind public keys to subjects
- DNSSEC
 - Uses signed DNSKEY and DS to connect zone keys
- The validated object differs
 - Certificate chain vs. DNS record

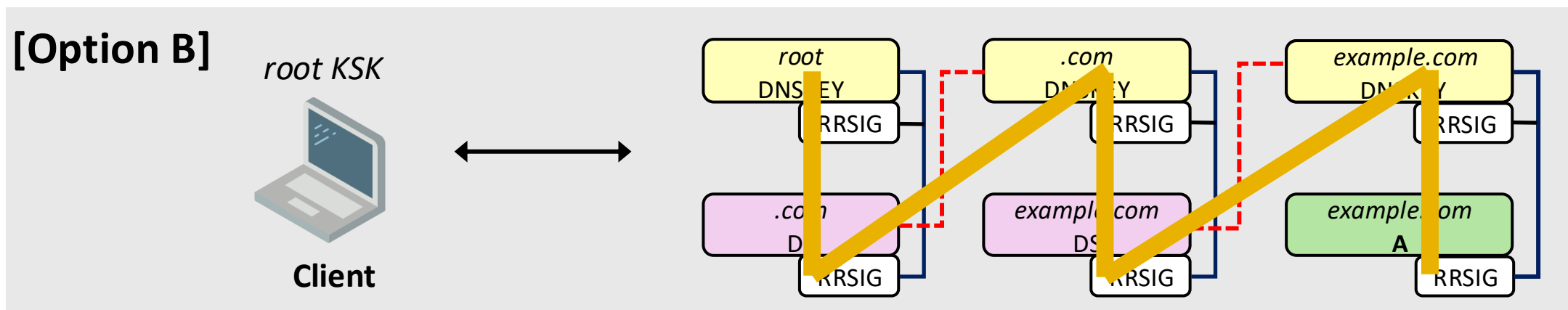
Where DNSSEC Validation Runs

- **[Option A]** a recursive resolver can check DNSSEC before returning data to clients
 - Requires trust in resolver
 - The resolver returns the validation result (e.g., Authenticated Data; AD flag) to the client



Where DNSSEC Validation Runs

- [Option A] a recursive resolver can check DNSSEC before returning data to clients
 - Requires trust in resolver
 - The resolver returns the validation result (e.g., Authenticated Data; AD flag) to the client
- **[Option B]** a client can perform its own checks using a local trust anchor and the required records



DNSSEC Operational Responsibilities

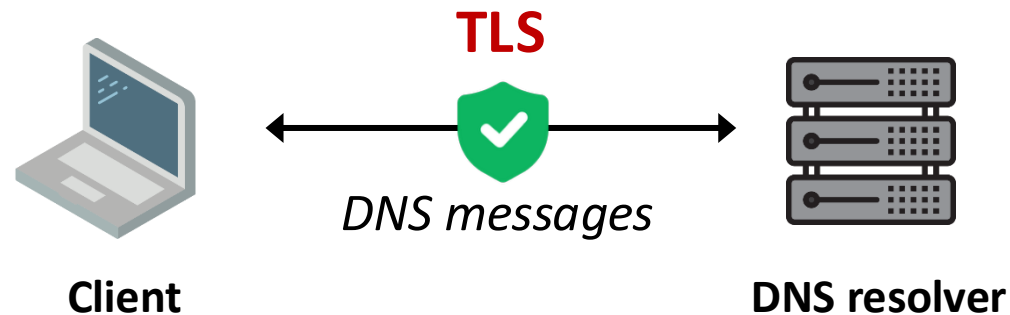
- For the correct DNSSEC support, each entity should perform its role correctly
- Zone operator
 - Protect signing keys, publish consistent records, and renew signatures
 - Publish DS that matches the intended child key and coordinate changes
- Resolver (or client validator)
 - Maintain trust anchors and validation policy
- Client
 - Handle resolution failure properly (e.g., reject the DNS records)

DNS over Encryption

- DNSSEC authenticates DNS data, but does not encrypt DNS messages
 - DNS queries and responses can still be observed on the network
- Encrypted DNS protects communication between a client and a resolver
 - Prevents on-path observers from reading or modifying DNS messages
 - Does not replace DNSSEC validation
 - Does not hide queries from the resolver itself
- Two common mechanisms
 - DNS over TLS (DoT)
 - DNS over HTTPS (DoH)

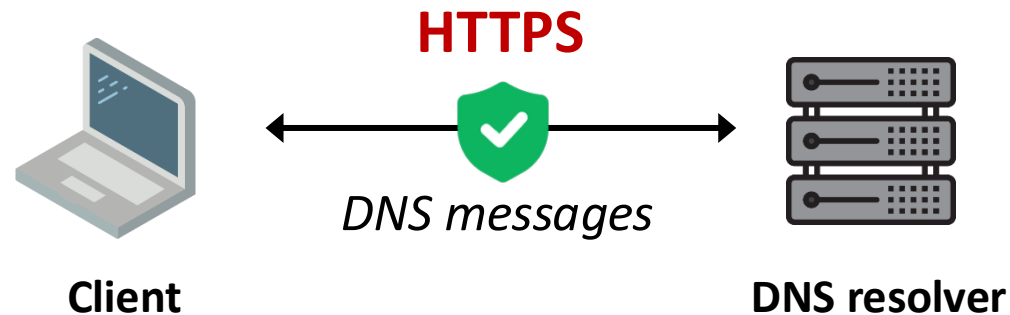
DNS over TLS (DoT)

- Protects DNS messages using a TLS connection
- DNS messages are carried directly over the TLS-protected connection
- Uses a dedicated port
 - Typically TCP port 853



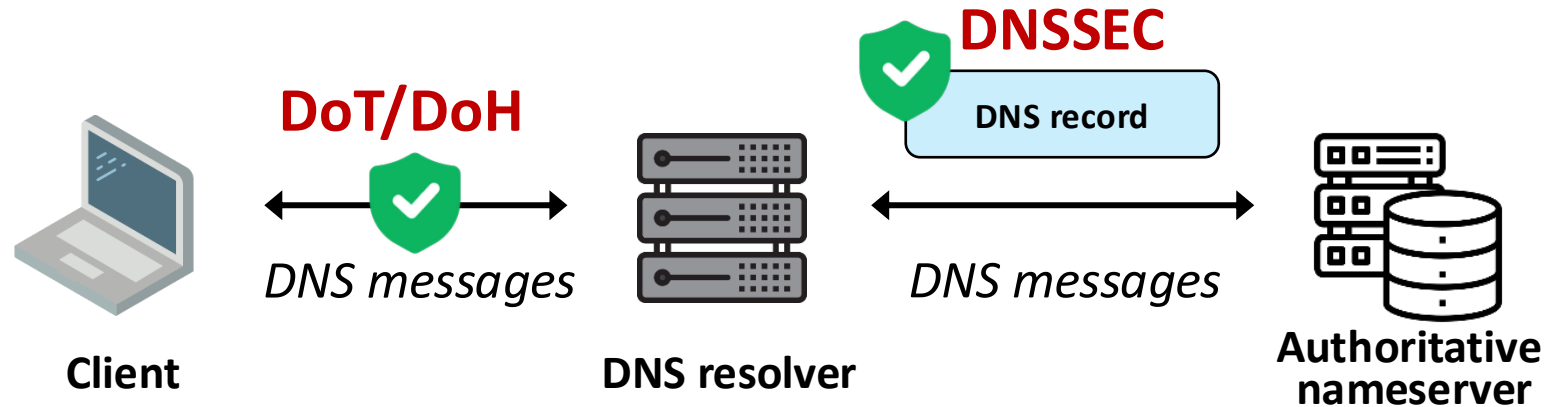
DNS over HTTPS (DoH)

- Protects DNS messages using HTTPS
- DNS queries and responses are carried inside HTTP messages
 - Typically uses HTTPS port 443
- Can share the same ordinary HTTPS web traffic
 - Blends with ordinary HTTPS traffic
 - Can provide additional privacy against on-path network observers



DNSSEC and Encrypted DNS

- DNSSEC
 - Authenticates DNS record data using the DNSSEC chain of trust
 - Does not encrypt DNS transport
- DoT / DoH
 - Protect DNS transport between the client and resolver
 - Provide confidentiality and integrity for that communication



Summary

- DNSSEC authenticates DNS data using digital signatures
 - DNSKEY, RRSIG, and DS records establish a chain of trust
- DNSSEC validation starts from a trusted root key
 - Validators verify the chain down to the target DNS RRset
- DNSSEC provides data origin authentication and integrity
 - But it does not provide confidentiality
- DoT/DoH encrypt DNS transport between a client and resolver